Name = Harsh Kumar Gola

CU24250165

BTech CSE 3rd Year SECTION - "B"

Roll No. :- 23

Experiment Name

Frequency Domain Image Filtering using Fourier Transform

==========

To implement image filtering in the frequency domain using Fourier Transform and analyze the effectiveness of frequency-based filtering techniques for noise removal, image enhancement, and feature preservation using Python and OpenCV.

Step 1: Import Required Libraries and upload / Select Image from Computer

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

try:
    from google.colab import files
    uploaded = files.upload()
    if not uploaded:
        print("No file uploaded.")
        exit()
    image_path = list(uploaded.keys())[0]
    print(f"File '{image_path}' uploaded successfully.")
except ImportError:
    # Fallback for local execution or non-Colab environments
    from tkinter import Tk, filedialog
    root = Tk()
    root.withdraw()

    image_path = filedialog.askopenfilename(
        title="Select an Image",
        filetypes=[
            ("Image Files", "*.jpg *.jpeg *.png *.bmp *.tif *.tiff"),
            ("All Files", "*.*")
        ]
    )

    if not image_path:
        print("No image selected.")
        exit()

image = cv2.imread(
image_path,
cv2.IMREAD_GRAYSCALE
)

if image is None:
    print(f"Error: Unable to load the selected image at '{image_path}'.")
    exit()

print("Image uploaded successfully.")
print("Selected Image:", image_path)
print("Image Shape:", image.shape)
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Screenshot 2025-03-01 190609.png to Screenshot 2025-03-01190609.png
File 'Screenshot 2025-03-01 190609.png' uploaded successfully.
Image uploaded successfully.
Selected Image: Screenshot 2025-03-01 190609.png
Image Shape: (1079, 1919)
```

Step 2: Compute Discrete Fourier Transform (DFT)

```
dft = cv2.dft(
    np.float32(image),
    flags=cv2.DFT_COMPLEX_OUTPUT
)

dft_shift = np.fft.fftshift(dft)

print("Discrete Fourier Transform (DFT) calculated successfully.")
print("Zero-frequency component shifted to the center.")
```

```
Discrete Fourier Transform (DFT) calculated successfully.
Zero-frequency component shifted to the center.
```

Step 3: Compute Discrete Fourier Transform (DFT)

```
dft = cv2.dft(
    np.float32(image),
    flags=cv2.DFT_COMPLEX_OUTPUT
)

# Shift zero-frequency component to the center
dft_shift = np.fft.fftshift(dft)

print("Discrete Fourier Transform (DFT) calculated successfully.")
print("Zero-frequency component shifted to the center.")
```

```
Discrete Fourier Transform (DFT) calculated successfully.
Zero-frequency component shifted to the center.
```

Step 4: Calculate Magnitude Spectrum

```
magnitude_spectrum = cv2.magnitude(
    dft_shift[:, :, 0],
    dft_shift[:, :, 1]
)

magnitude_spectrum = 20 * np.log(
```

```
    magnitude_spectrum + 1
)

print("Magnitude spectrum calculated successfully.")
```

```
Magnitude spectrum calculated successfully.
```

Step 5: Create Low-Pass Filter

```
rows, cols = image.shape

crow, ccol = rows // 2, cols // 2

# Radius of the filter
radius = 50

low_pass_mask = np.zeros(
    (rows, cols, 2),
    np.uint8
)

cv2.circle(
    low_pass_mask,
    (ccol,  crow),
    radius,
    (1, 1),
    -1
)

print("Low-Pass Filter created successfully.")
```

```
Low-Pass Filter created successfully.
```

Step 6: Apply Low-Pass Filter

```
low_pass_dft = dft_shift * low_pass_mask

# Shift back
low_pass_dft = np.fft.ifftshift(
    low_pass_dft
)

# Inverse Fourier Transform
low_pass_image = cv2.idft(
    low_pass_dft
)

low_pass_image = cv2.magnitude(
    low_pass_image[:, :, 0],
    low_pass_image[:, :, 1]
)

# Normalize
low_pass_image = cv2.normalize(
    low_pass_image,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

low_pass_image = np.uint8(
    low_pass_image
)

print("Low-Pass Filtering completed successfully.")
```

```
Low-Pass Filtering completed successfully.
```

Step 7: Create High-Pass Filter

```
high_pass_mask = np.ones(
    (rows, cols, 2),
    np.uint8
)

cv2.circle(
    high_pass_mask,
    (ccol,   crow),
    radius,
    (0, 0),
    -1
)

print("High-Pass Filter created successfully.")
```

```
High-Pass Filter created successfully.
```

Step 8: Apply High-Pass Filter

```
high_pass_dft = dft_shift * high_pass_mask

# Shift back
high_pass_dft = np.fft.ifftshift(
    high_pass_dft
)

# Inverse Fourier Transform
high_pass_image = cv2.idft(
    high_pass_dft
)

high_pass_image = cv2.magnitude(
    high_pass_image[:, :, 0],
    high_pass_image[:, :, 1]
)

# Normalize
high_pass_image = cv2.normalize(
    high_pass_image,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)
```

```
high_pass_image = np.uint8(
    high_pass_image
)

print("High-Pass Filtering completed successfully.")
```

```
High-Pass Filtering completed successfully.
```

Step 9: Display Results

```
plt.figure(figsize=(12, 8))

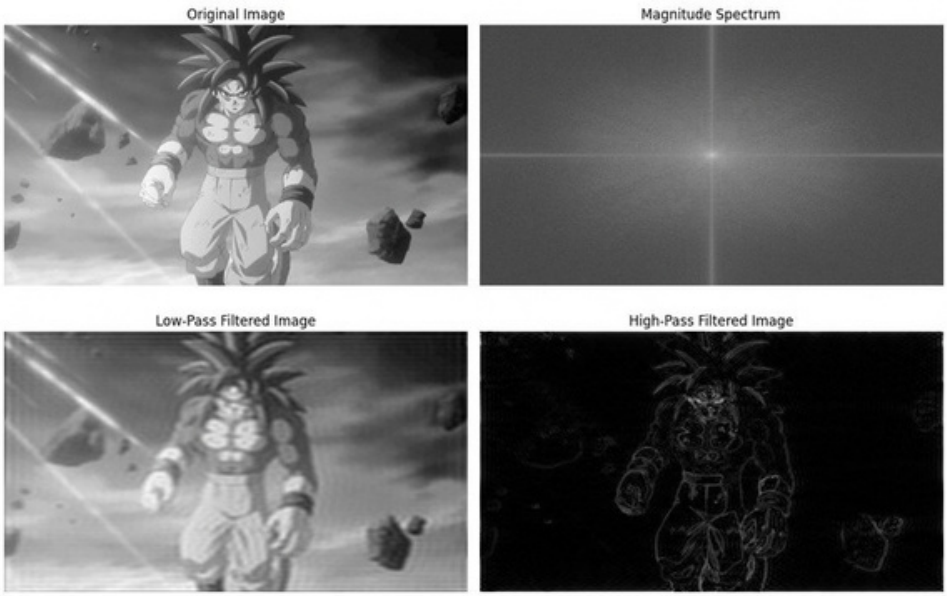
plt.subplot(2, 2, 1)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(low_pass_image, cmap='gray')
plt.title("Low-Pass Filtered Image")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(high_pass_image, cmap='gray')
plt.title("High-Pass Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 10: Final Output

```
print("\n========== FREQUENCY DOMAIN FILTERING ==========")

print("Original Image Shape:", image.shape)

print("DFT      Computation:      Completed")
print("Magnitude     Spectrum:    Generated")
print("Low-Pass     Filtering:    Completed")
print("High-Pass    Filtering:    Completed")
print("Inverse Fourier Transform: Completed")

print("\nObservation:")
print("Low-Pass Filter reduces high-frequency components and")
print("produces a smoother image with reduced noise.")

print("\nHigh-Pass Filter suppresses low-frequency components")
print("and enhances edges and fine details.")

print("\nExperiment Completed Successfully.")
```

```
========== FREQUENCY DOMAIN FILTERING ==========
Original Image Shape: (1079, 1919)
DFT Computation: Completed
Magnitude Spectrum: Generated
Low-Pass Filtering: Completed
High-Pass Filtering: Completed
Inverse Fourier Transform: Completed



Observation:
Low-Pass Filter reduces high-frequency components and
produces a smoother image with reduced noise.


High-Pass Filter suppresses low-frequency components
and enhances edges and fine details.


Experiment Completed Successfully.
```

== EXPERIMENT 4 - ANSWERS TO QUESTIONS QUESTION 1:

What is the Fourier Transform? Why is it important in digital image processing?

Fourier Transform is a mathematical technique that converts an image from the spatial domain into the frequency domain.

It represents an image using different frequency components.

Low-frequency components represent smooth regions and gradual changes in intensity.

High-frequency components represent edges, fine details, and noise.

Fourier Transform is important because it allows us to manipulate different frequency components independently.

It is widely used for image filtering, enhancement, restoration, noise removal and feature extraction.

QUESTION 2: Differentiate between the spatial domain and the frequency domain.
Spatial Domain: Image processing is performed directly on the pixels of an image.
Examples: Blurring, sharpening, thresholding and edge detection.
Frequency Domain: Image is first converted into frequency components using Fourier Transform.
Filtering is then performed on these frequency components.
Main Difference:
Spatial Domain -> Operations are performed directly on pixels. Frequency Domain -> Operations are performed on frequencies.
QUESTION 3: What is the significance of the Discrete Fourier Transform (DFT) in image processing?
DFT converts a digital image from the spatial domain into its frequency representation.
It helps identify low-frequency and high-frequency components present in an image.
DFT is significant because it allows efficient implementation of frequency domain filtering.
It is commonly used for:
Noise removal Image enhancement Image restoration Edge enhancement Feature extraction QUESTION 4: Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.
In the Fourier Transform, the zero-frequency component is normally located at the corner of the frequency spectrum.
The fftshift() function moves the zero-frequency component to the center of the spectrum.
This makes the frequency spectrum easier to visualize and analyze.
After shifting:
Center -> Low-frequency components Outer regions -> High-frequency components
Therefore, shifting makes frequency domain filtering easier to design and understand.
QUESTION 5: Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.
Low-Pass Filter:
A Low-Pass Filter allows low-frequency components to pass while suppressing high-frequency components.
It produces a smoother image and can reduce noise.
Applications:
Noise reduction Image smoothing Blur reduction of small variations High-Pass Filter:
A High-Pass Filter allows high-frequency components to pass while suppressing low-frequency components.
It enhances edges and fine details.
Applications:
Edge enhancement Sharpening Feature extraction Main Difference:
Low-Pass Filter -> Smoothing and noise reduction High-Pass Filter -> Edge and detail enhancement
QUESTION 6: What is the role of the Inverse Fourier Transform (IDFT) in image reconstruction?
The Inverse Fourier Transform converts the filtered frequency-domain representation back into the spatial domain.
After applying a frequency filter, the image exists in the frequency domain.
IDFT reconstructs the image so that it can be viewed as a normal spatial-domain image.
Therefore:
DFT -> Spatial Domain to Frequency Domain IDFT -> Frequency Domain to Spatial Domain
QUESTION 7: Why is frequency domain filtering preferred for certain image enhancement tasks?
Frequency domain filtering is preferred for certain tasks because it allows selective manipulation of frequency components.
Low frequencies and high frequencies can be controlled independently.
It is especially useful for:
Noise removal Image smoothing Edge enhancement Image restoration Feature extraction Frequency domain methods can also be computationally efficient when implemented using Fast Fourier Transform (FFT).
QUESTION 8: Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.
Four real-world applications are:
Medical Image Enhancement: Fourier Transform can be used to improve medical images such as MRI and CT images.
Satellite Image Analysis: Frequency filtering can help remove noise and enhance important structures in satellite images.
Image Restoration: It can be used to reduce noise and recover useful information from degraded images.
Biometric Systems: Frequency-based processing can help enhance features in fingerprint, face and other biometric images.
QUESTION 9: Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.
Spatial Domain Filtering:
Operations are performed directly on image pixels.
It is simple and suitable for local operations such as:
Blur Sharpening Edge detection Thresholding Frequency Domain Filtering:
The image is converted into the frequency domain first.
It is useful for operations involving specific frequency components such as:
Noise removal Smoothing Edge enhancement Image restoration Frequency domain filtering can be computationally efficient for large images and large convolution filters when FFT-based methods are used.
QUESTION 10: How does frequency domain filtering improve the performance of image restoration and feature extraction techniques?
Frequency domain filtering allows unwanted frequency components to be selectively removed or enhanced.
In image restoration, unwanted noise can be reduced by suppressing the corresponding high-frequency components.
In feature extraction, high-frequency components containing edges and fine details can be enhanced using High-Pass Filters.
Therefore, frequency domain filtering can:
Reduce image noise. Improve image quality. Enhance edges. Preserve important features. Improve the quality of input images used by computer vision algorithms. This can help improve the performance of image restoration and feature extraction techniques.